Table-Based Encodings for Conway's Doomsday Algorithm: Vectorized Doomsdays and Doomyears

Thomas Wollin
Toronto, Ontario, Canada
github.com/Nillows

March 28, 2026

Abstract

Conway's Doomsday Algorithm (1973) determines the day of the week for any date in the Gregorian calendar via three additive components: a century anchor, a year offset, and a month–day offset. The century anchor is a fixed four-entry table. The other two components require live arithmetic: the year offset demands computing $y + \lfloor y/4 \rfloor \pmod 7$, and the month–day offset requires a subtraction that can produce negative intermediate values. We present two new encoding schemes that replace both arithmetic steps with structured table lookups.

The first, *vectorized doomsdays*, re-encodes each month's doomsday date as a two-digit number whose tens and units digits represent the backward and forward gaps (respectively) from the nearest multiples-of-seven month anchors. A directional crossing rule (the "square knot rule") pairs the target date's gap with the opposite-direction digit, reducing the month–day offset to a single-digit addition.

The second, *Doomyears*, encodes the year-offset function as a navigational lookup exploiting the 28-year periodicity of the Gregorian weekday cycle.

We prove correctness, establish self-verification properties, analyse the internal structure of both encodings, and compare the cognitive complexity of the table-based system against the standard arithmetic method.

1 Introduction

The Doomsday Algorithm, introduced by Conway [3] and described in [1], computes the day of the week for any date as:

$$\text{day of week} \equiv c + \omega(y) + \delta(m, d) \pmod 7, \tag{1}$$

where c is the century anchor, $\omega(y) = (y + \lfloor y/4 \rfloor) \bmod 7$ is the year offset, and $\delta(m, d)$ is the signed distance from the target date to the month's doomsday date.

Step c is a four-entry table. Steps ω and δ require real-time arithmetic. We replace both with pre-computed encodings.

2 Part 1: Vectorized doomsdays

2.1 Month anchors and gap structure

Definition 1. The *month anchors* are $A = \{7, 14, 21, 28\}$, the multiples of 7 occurring as valid dates in every month.

Definition 2. For a date t with $7 \leq t \leq 31$, let $a^-(t) = \max\{a \in A : a \leq t\}$ and $a^+(t) = \min\{a \in A : a \geq t\}$. Define the *forward gap* $g_F(t) = t - a^-(t)$ and the *backward gap* $g_B(t) = a^+(t) - t$. For dates $1 \leq t \leq 6$, we set $a^+(t) = 7$ and $g_B(t) = 7 - t$, with $g_F(t)$ undefined (no valid anchor below). For dates $29 \leq t \leq 31$, we have $a^-(t) = 28$ and $g_F(t) = t - 28$, with $g_B(t)$ undefined (no valid anchor in A at or above t). In both boundary cases the square knot rule must use the direction that is defined.

Proposition 3. *For any date t with $7 \leq t \leq 28$ not on an anchor, $g_F(t) + g_B(t) = 7$. When $t \in A$, $g_F(t) = g_B(t) = 0$.*

Proof. If t lies strictly between consecutive anchors a^- and a^+, then $a^+ - a^- = 7$, so $g_F + g_B = (t - a^-) + (a^+ - t) = 7$. If $t \in A$, then $a^- = a^+ = t$. □

2.2 Construction of the vectorized doomsdays

Definition 4. Let d_m denote the traditional doomsday date for month m. The *vectorized doomsday* for month m is

$$V_m = 10 \cdot g_B(d_m) + g_F(d_m).$$

The explicit values are:

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
d_m	3	28	7	4	9	6	11	8	5	10	7	12
V_m	43	00	00	34	52	16	34	61	25	43	00	25

Table 1: Traditional doomsday dates and vectorized doomsdays. Leap year: Jan $d_m = 4 \to V = 34$; Feb $d_m = 29 \to V = 61$.

Proposition 5 (Digit-sum property)**.** *For every vectorized doomsday V_m, either both digits are 0 or the digits sum to 7.*

Proof. Immediate from Proposition 3: the tens digit is $g_B(d_m)$ and the units digit is $g_F(d_m)$. □

Proposition 6 (Spatial encoding)**.** *The tens (left) digit of V_m encodes the leftward (backward) gap; the units (right) digit encodes the rightward (forward) gap. The direction a digit represents matches the direction one reads to find it.*

2.3 The square knot rule

Theorem 7 (Correctness of the month–day offset)**.** *Let t be a target date in month m, with forward gap $g_F(t)$ from its nearest lower anchor. Let $V_m = 10b + f$ where $b = g_B(d_m)$ and $f = g_F(d_m)$. Then*

$$t - d_m \equiv g_F(t) + b \pmod 7.$$

Similarly, if the backward gap $g_B(t)$ is used instead, then

$$d_m - t \equiv g_B(t) + f \pmod 7.$$

Proof. Write $t = a^-(t) + g_F(t)$ where $a^-(t)$ is a multiple of 7. Write $d_m = a^+(d_m) - g_B(d_m) = a^+(d_m) - b$ where $a^+(d_m)$ is a multiple of 7. Then $t - d_m = a^-(t) + g_F(t) - a^+(d_m) + b \equiv g_F(t) + b \pmod 7$, since both $a^-(t)$ and $a^+(d_m)$ are multiples of 7.

The second identity follows by symmetry: $d_m - t \equiv g_B(t) + f \pmod 7$ by the same argument with roles exchanged. □

Remark 8. The forward-gap case gives a result in the context of "days after doomsday" (added to the year's doomsday), while the backward-gap case gives "days before doomsday" (subtracted). The choice of direction at the target date determines both which digit is selected *and* the sign of the result — these are not independent decisions.

Remark 9 (Why same-direction pairing requires subtraction)*.* One might ask whether the encoding could instead pair same directions — forward with forward, backward with backward — eliminating the crossing rule. Same-direction pairing is valid, but only under *subtraction*: $g_F(t) - g_F(d_m) \equiv t - d_m \pmod 7$, since both reference anchors are multiples of 7 and cancel. Under *addition*, $g_F(t) + g_F(d_m) \equiv t + d_m \pmod 7$, which differs from the correct value $t - d_m$ by $2d_m$. One could reverse the digit positions in V_m to support same-direction subtraction, but this reintroduces the negative intermediate values that the encoding was designed to eliminate. The cross-direction rule is therefore the structural cost of an all-addition, all-non-negative system.

Corollary 10 (Self-consistency)**.** *For every month m, running the traditional doomsday date d_m through its own vectorized doomsday yields offset* 0*.*

Proof. $g_F(d_m) + g_B(d_m) = 7 \equiv 0 \pmod 7$ (or $0 + 0 = 0$ when d_m is an anchor). □

2.4 Structure of the code space

Proposition 11. *Exactly four distinct non-zero digit pairs appear among the vectorized doomsdays: $\{1, 6\}$, $\{2, 5\}$, $\{3, 4\}$, and $\{0, 0\}$. Each non-zero pair appears in both orientations (e.g.,* 16 *and* 61*).*

Proof. Since months have at most 31 days and the highest anchor is 28, the maximum forward gap from anchor 28 is $31 - 28 = 3$. Thus $g_F \in \{0, 1, 2, 3\}$ for any date in such a month. Since $g_F + g_B = 7$ (or both $= 0$), the only possible pairs (g_B, g_F) are $(0, 0)$, $(6, 1)$, $(5, 2)$, and $(4, 3)$ — that is, the unordered pairs $\{0, 0\}$, $\{1, 6\}$, $\{2, 5\}$, and $\{3, 4\}$. Inspection of Table 1 confirms that each of these four pairs is realised by at least one traditional doomsday date, and each non-zero pair appears in both orientations. □

2.5 Equivalence classes of anchor-date systems

The vectorized doomsday construction does not depend on Conway's specific choice of anchor dates. Any set of twelve dates (one per month) that share a common weekday within each year can serve as input, and the construction produces a valid code table. We now classify all such systems.

Definition 12. Two anchor-date systems $\{d_m\}$ and $\{d'_m\}$ are *equivalent* if $d_m \equiv d'_m \pmod 7$ for every month m.

Since the vectorized doomsday V_m depends only on $d_m \bmod 7$, equivalent systems produce identical code tables.

Proposition 13 (Seven equivalence classes)**.** *There are exactly* 7 *equivalence classes of same-weekday anchor-date systems, indexed by a uniform offset* $k \in \{0, 1, \ldots, 6\}$ *from Conway's doomsday dates. System* k *has anchor-date residues* $d_m + k \pmod 7$ *and century anchors* $c - k \pmod 7$*, where* c *is Conway's century anchor.*

Proof. Let $\{d_m\}$ be Conway's doomsday dates and $\{d'_m\}$ any other same-weekday system. In a given year, all d_m share a weekday W and all d'_m share a weekday W'. Within each month m, the weekday of the nth is determined by $n \bmod 7$ (up to a month-specific constant), so $d'_m \equiv d_m + (W' - W) \pmod 7$ for every m. Setting $k = W' - W \bmod 7$ gives a uniform shift. Since the overall algorithm must yield the same final weekday, the century anchor absorbs the complementary shift: $c' = c - k \pmod 7$. □

Proposition 14 (Invariance of month groupings and code vocabulary)**.** *All seven systems produce the same set of codes* $\{00, 16, 25, 34, 43, 52, 61\}$ *and preserve the same month groupings (which months share a code). Incrementing* k *by* 1 *cycles every code one step through the rotation* $00 \to 61 \to 52 \to 43 \to 34 \to 25 \to 16 \to 00$.

Proof. The code for month m in system k depends only on $(d_m + k) \bmod 7$. Incrementing k uniformly shifts every month's residue, applying the same cyclic permutation of codes to all months simultaneously. Since months with $d_m \equiv d_{m'} \pmod 7$ undergo the same shift, they always receive the same code. □

The seven month groupings, determined by the residue classes of Conway's doomsday dates, are:

$d_m \bmod 7$	Months	Size
0	Feb, Mar, Nov	3
1	Aug	1
2	May	1
3	Jan, Oct	2
4	Apr, Jul	2
5	Sep, Dec	2
6	Jun	1

Proposition 15 (Optimality of Conway's system)**.** *Among the seven equivalence classes, Conway's ($k = 0$) is the unique class that places the zero code* 00 *on the largest month group (February, March, November — three months). Every other class places* 00 *on a group of size at most* 2*.*

Proof. The residue-0 group {Feb, Mar, Nov} has size 3; all other residue classes have size ≤ 2. Only $k = 0$ maps residue 0 to code 00. □

Remark 16. Each equivalence class admits valid anchor dates — Wang's null-days ($k = 5$) demonstrate this with dates such as Feb 12 and Nov 12. The optimality of $k = 0$ is not a question of feasibility but of *efficiency*: three zero-months versus at most two for any other class. Code 00 is the most efficient to use (the month contributes nothing to the sum, so the month–day step reduces to a single gap measurement), making the number of zero-months a natural measure of system quality.

Remark 17. Wang's null-days algorithm [5] uses anchor dates 1/1, 3/5, 5/7, 7/9, 9/3, 2/12, 12/10, 10/8, 8/6, 6/4, 4/2, 11/12, which belong to the $k = 5$ equivalence class: every Wang date satisfies $d'_m \equiv d_m + 5 \pmod 7$. The resulting vectorized code table is $61, 25, 25, 52, 00, 34, 52, 16, 43, 61, 25, 43$ — a rotation of Conway's with only one zero-month (May) instead of three, and century anchors shifted by -5 (e.g. the 2000s anchor becomes $(2 - 5) \bmod 7 = 4$).

3 Part 2: Doomyears

3.1 Periodicity

Theorem 18 (28-year periodicity)**.** $\omega(y + 28) = \omega(y)$ *for all* $y \geq 0$*.*

Proof. $\lfloor (y + 28)/4 \rfloor = \lfloor y/4 \rfloor + 7$, so the total increases by $28 + 7 = 35 \equiv 0 \pmod 7$. □

Corollary 19. *The anchor years* $0, 28, 56, 84$ *satisfy* $\omega(y) = 0$*.*

3.2 Packed-number sequences

Definition 20. For $d \in \{0, \ldots, 15\}$, define $f(d) = \omega(d)$, $b(d) = \omega(28 - d)$, and the packed numbers $F_d = 10d + f(d)$, $B_d = 10d + b(d)$.

Definition 21. The *Doomyear* at distance d is $D_d = 100d + 10 \cdot b(d) + f(d)$, encoding both directions in a single number: distance, backward digit, forward digit.

d	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
F_d	00	11	22	33	45	56	60	71	83	94	105	116	121	132	143	154
B_d	00	15	24	33	42	50	66	75	84	92	101	110	126	134	143	152
D_d	00	151	242	333	425	506	660	751	843	924	1015	1106	1261	1342	1433	1524

Table 2: Forward, backward, and interleaved Doomyear sequences.

3.3 Structural properties

Proposition 22 (Leap-year stumble)**.** $\Delta_f(d) = f(d) - f(d-1) \pmod 7$ *equals* 2 *when* $4 \mid d$ *and* 1 *otherwise.*

Proof. $\Delta_f(d) = 1 + \lfloor d/4 \rfloor - \lfloor (d-1)/4 \rfloor$. The floor difference is 1 when $4 \mid d$, else 0. □

Proposition 23 (Complementarity)**.** *For all* $d \in \{0, \ldots, 15\}$,

$$f(d) + b(d) \equiv \begin{cases} 0 \pmod 7 & \textit{if } 4 \mid d, \\ 6 \pmod 7 & \textit{otherwise.} \end{cases}$$

Proof. $f(d)+b(d) = d+\lfloor d/4 \rfloor + (28-d) + \lfloor (28-d)/4 \rfloor \pmod 7 = 28 + \lfloor d/4 \rfloor + \lfloor (28-d)/4 \rfloor \pmod 7$. Writing $d = 4q + r$: when $r = 0$, $\lfloor d/4 \rfloor + \lfloor (28-d)/4 \rfloor = q + (7-q) = 7 \equiv 0$; when $r > 0$, $= q + (6-q) = 6$. Since $28 \equiv 0$, the result is 0 or 6. □

Remark 24. Proposition 23 shows that the Doomyear digit-sum property is *not* identical to that of the vectorized doomsdays. The month codes always have digits summing to 7 (or both 0), a consequence of the fixed seven-unit span between consecutive anchors. The Doomyear directional digits sum to $0 \pmod 7$ (i.e. both 0 or summing to 7) only at multiples-of-4 distances; at all other distances they sum to $6 \pmod 7$. The structural parallel between the two encodings holds in form — directional digits packed by position — but the verification guarantee is weaker for Doomyears and should not be overstated.

4 Cognitive complexity comparison

The operation count understates the difference. The standard method's operations are serial (each depends on the prior result) and involve intermediate values up to 124. The shortcut's operations involve no value larger than 6 and the two steps (month and year) are fully independent.

5 Memorisation burden

The complete system requires:

- 4 century anchors (Conway's original).
- 12 vectorized doomsday codes (4 distinct non-zero pairs in two orientations, plus three zero-months).
- 16 Doomyears (or equivalently, 2×16 directional packed numbers).

Structural redundancy reduces the effective burden: month code digits sum to 7; mirror pairs (16/61, 25/52, 34/43) halve the distinct codes; the leap-year stumble rhythm makes the year sequences reconstructable from checkpoints; and Doomyear directional digits sum to 7 (or 0), enabling derivation of one from the other.

Operation	**Standard**	**Wollin Shortcut**
Year-offset step		
Integer division ($\lfloor y/4 \rfloor$)	1	0
Multi-digit addition	1	0
Modular reduction (mod 7, large)	1	0
Small subtraction (distance to anchor)	0	1
Table recall	0	1
Month–day step		
Subtraction (target − doomsday)	1	0
Sign correction / mod 7	1	0
Gap measurement (to month anchor)	0	1
Digit selection + single-digit add	0	1
Total operations	5	4
Operation dependency	Serial	Independent
Max intermediate value	124	6
Divisions required	1	0
Mod reductions on large numbers	2	0

Table 3: Mental operations for the two variable steps combined.

6 Historical context

The 28-year periodicity is classical (Carroll [2]). Various year-offset shortcuts exist, including Conway's "odd+11" rule [1]. The month–day step has historically been handled by memorising the traditional doomsday dates and performing a subtraction.

The vectorized doomsday encoding — representing each month's doomsday information as a directionally-aware, self-verifying two-digit code derived from multiple-of-7 anchors — appears to be new. The structural parallel between the month codes and the Doomyear codes (both using complementary digit pairs, spatial direction encoding, and digit-sum verification) unifies the two variable steps of the algorithm under a single design principle.

The classification of anchor-date systems into seven equivalence classes (Section 2.5) and the proof that Conway's class is uniquely optimal for the vectorized encoding also appear to be new. Wang's null-days [5], which belong to a different equivalence class ($k = 5$), confirm that the encoding technique generalises beyond Conway's specific dates, while the optimality result (Proposition 15) shows that Conway's dates yield the best code table among all seven classes.

References

[1] E. R. Berlekamp, J. H. Conway, and R. K. Guy, *Winning Ways for Your Mathematical Plays*, vol. 2, Academic Press, 1982.

[2] L. Carroll, "To find the day of the week for any given date," *Nature*, vol. 35, no. 892, 1887, p. 517.

[3] J. H. Conway, "Tomorrow is the day after doomsday," *Eureka*, vol. 36, 1973, pp. 28–31.

[4] C. Fong and M. K. Walters, "Methods for accelerating Conway's Doomsday algorithm," *ICERM Technical Report*, 2010.

[5] X.-S. Wang, "Calculating the day of the week: null-days algorithm," *arXiv preprint* arXiv:1404.2510, 2014.